

Android Error Handling: Complete Guide

Comprehensive Reference for Exception Handling, Result Wrappers, and Error Management

Table of Contents

1. [What is Error Handling?](#)
 2. [Types of Errors in Android](#)
 3. [Try-Catch Basics](#)
 4. [Result Wrapper Pattern](#)
 5. [Sealed Class Approach](#)
 6. [Flow Error Handling](#)
 7. [Global Exception Handler](#)
 8. [Best Practices](#)
 9. [Real-World Examples](#)
 10. [Testing Error Handling](#)
 11. [Comparison Table](#)
 12. [Anti-Patterns to Avoid](#)
-

1. What is Error Handling?

Definition

Error handling is the process of responding to and recovering from error conditions in your application. In Android, this means catching exceptions, managing failures gracefully, and providing meaningful feedback to users.

Why Error Handling Matters

Without proper error handling:

- ☐ App crashes unexpectedly
- ☐ Users see cryptic "App has stopped" messages
- ☐ Data loss and poor user experience
- ☐ Difficult to debug production issues

With proper error handling:

- ✓ Graceful degradation (app continues working)
- ✓ Clear error messages for users
- ✓ Logged errors for debugging
- ✓ Better app ratings and user retention

Core Principles

- **Fail gracefully** - Don't crash, show meaningful errors
 - **Log for debugging** - Record errors for analysis
 - **User-friendly messages** - "Something went wrong" not "NullPointerException"
 - **Recovery mechanisms** - Retry, fallback, or alternative flows
 - **Specific exception handling** - Catch what you expect
-

2. Types of Errors in Android

Checked Exceptions

Definition: Exceptions that must be handled at compile time (Java only - Kotlin doesn't have checked exceptions).

Examples:

- `IOException` - File/network operations
- `SQLException` - Database operations
- `ClassNotFoundException` - Reflection operations

Java Example:

```
// MUST handle at compile time
public void readFile() throws IOException {
```

```
FileInputStream file = new FileInputStream("data.txt");  
// ... code  
}
```

```
// Or catch it  
public void readFile() {  
    try {  
        FileInputStream file = new FileInputStream("data.txt");  
    } catch (IOException e) {  
        Log.e("TAG", "File error", e);  
    }  
}
```

Kotlin Example:

```
// No forced handling (Kotlin treats all as unchecked)  
fun readFile() {  
    val file = FileInputStream("data.txt")  
    // ... code  
}
```

```
// But you SHOULD still handle  
fun readFile() {  
    try {  
        val file = FileInputStream("data.txt")  
    } catch (e: IOException) {  
        Log.e("TAG", "File error", e)  
    }  
}
```

Unchecked Exceptions (Runtime Exceptions)

Definition: Exceptions that occur at runtime and aren't forced to be handled.

Common Android Examples:

Exception	Cause
NullPointerException	Accessing null object
IndexOutOfBoundsException	Invalid array/list index
IllegalArgumentException	Invalid method argument
NumberFormatException	Invalid string-to-number conversion
ClassCastException	Invalid type casting
ActivityNotFoundException	Starting non-existent Activity
NetworkOnMainThreadException	Network call on UI thread

Example:

```
// NullPointerException
val name: String? = null
val length = name.length // CRASH!
```

```
// Solution
val length = name?.length ?: 0 // Safe call
```

Android-Specific Errors

1. ANR (Application Not Responding)

- Cause: UI thread blocked for >5 seconds
- Solution: Move long operations to background threads

2. OutOfMemoryError

- Cause: Loading large bitmaps, memory leaks
- Solution: Use BitmapFactory options, proper lifecycle management

3. NetworkOnMainThreadException

- Cause: Network calls on main thread (Android 3.0+)
- Solution: Use coroutines, RxJava, or WorkManager

4. TransactionTooLargeException

- Cause: Intent/Bundle data >1MB
- Solution: Pass IDs instead of full objects

3. Try-Catch Basics

Basic Syntax

Kotlin:

```
try {  
    // Code that might throw exception  
    val result = riskyOperation()  
} catch (e: SpecificException) {  
    // Handle specific exception  
    Log.e("TAG", "Specific error", e)  
} catch (e: Exception) {  
    // Handle any other exception  
    Log.e("TAG", "General error", e)  
} finally {  
    // Always executes (cleanup code)  
    closeResources()  
}
```

Java:

```
try {  
    // Code that might throw exception  
    int result = riskyOperation();  
} catch (SpecificException e) {  
    // Handle specific exception  
    Log.e("TAG", "Specific error", e);  
} catch (Exception e) {  
    // Handle any other exception  
    Log.e("TAG", "General error", e);  
} finally {  
    // Always executes (cleanup code)  
    closeResources();  
}
```

Example: Network Request

Kotlin:

```
fun fetchUserData(userId: String): User? {  
    return try {  
        val response = api.getUser(userId)
```

```

        if (response.isSuccessful) {
            response.body()
        } else {
            Log.e("API", "Error: ${response.code()}")
            null
        }
    } catch (e: IOException) {
        Log.e("API", "Network error", e)
        null
    } catch (e: HttpException) {
        Log.e("API", "HTTP error: ${e.code()}", e)
        null
    } catch (e: Exception) {
        Log.e("API", "Unknown error", e)
        null
    }
}

```

Java:

```

public User fetchUserData(String userId) {
    try {
        Response<User> response = api.getUser(userId).execute();
    }
}

```

```

        if (response.isSuccessful()) {
            return response.body();
        } else {
            Log.e("API", "Error: " + response.code());
            return null;
        }
    } catch (IOException e) {
        Log.e("API", "Network error", e);
        return null;
    } catch (HttpException e) {
        Log.e("API", "HTTP error: " + e.code(), e);
        return null;
    } catch (Exception e) {
    }
}

```

```
Log.e("API", "Unknown error", e);
return null;
}
```

```
}
```

Example: Safe Division

Kotlin:

```
fun safeDivide(a: Int, b: Int): Int? {
    return try {
        a / b
    } catch (e: ArithmeticException) {
        Log.e("Math", "Division by zero", e)
        Toast.makeText(context, "Cannot divide by zero",
            LENGTH_SHORT).show()
        null
    }
}

// Usage
val result = safeDivide(10, 0) ?: 0 // Returns 0 if error
```

Java:

```
public Integer safeDivide(int a, int b) {
    try {
        return a / b;
    } catch (ArithmeticException e) {
        Log.e("Math", "Division by zero", e);
        Toast.makeText(context, "Cannot divide by zero",
            Toast.LENGTH_SHORT).show();
        return null;
    }
}

// Usage
Integer result = safeDivide(10, 0);
if (result == null) {
    result = 0;
}
```

Multiple Catch Blocks

Kotlin:

```
fun parseData(input: String) {
    try {
        val number = input.toInt()
        val file = File("data.txt").readText()
        processData(number, file)
    } catch (e: NumberFormatException) {
        Log.e("TAG", "Invalid number format", e)
        showError("Please enter a valid number")
    } catch (e: FileNotFoundException) {
        Log.e("TAG", "File not found", e)
        showError("Data file is missing")
    } catch (e: IOException) {
        Log.e("TAG", "File read error", e)
        showError("Cannot read data file")
    } catch (e: Exception) {
        Log.e("TAG", "Unexpected error", e)
        showError("Something went wrong")
    }
}
```

Java (Multi-catch - Java 7+):

```
public void parseData(String input) {
    try {
        int number = Integer.parseInt(input);
        String fileContent = new Scanner(new File("data.txt"))
            .useDelimiter("\\Z").next();
        processData(number, fileContent);
    } catch (NumberFormatException e) {
        Log.e("TAG", "Invalid number format", e);
        showError("Please enter a valid number");
    } catch (FileNotFoundException | IOException e) {
        // Multi-catch: handle both with same code
        Log.e("TAG", "File error", e);
        showError("Cannot read data file");
    } catch (Exception e) {
        Log.e("TAG", "Unexpected error", e);
        showError("Something went wrong");
    }
}
```



```
}  
}
```

Kotlin runCatching

Modern Kotlin approach:

```
val result = runCatching {  
    riskyOperation()  
}.onSuccess { data ->  
    println("Success: $data")  
}.onFailure { exception ->  
    Log.e("TAG", "Error", exception)  
}.getOrNull()
```

```
// Or get with default  
val value = runCatching {  
    "123".toInt()  
}.getOrDefault(0)
```

```
// Or map result  
val mapped = runCatching {  
    fetchData()  
}.map { data ->  
    data.transform()  
}.getOrNull()
```

Kotlin runCatching with custom handling:

```
fun loadUser(userId: String): User? {  
    return runCatching {  
        api.getUser(userId)  
    }.onFailure { e ->  
        when (e) {  
            is IOException -> {  
                Log.e("API", "Network error", e)  
                showToast("No internet connection")  
            }  
            is HttpException -> {  
                Log.e("API", "Server error: ${e.code()}", e)  
                showToast("Server error")  
            }  
        }  
    }  
}
```

```
else -> {  
    Log.e("API", "Unknown error", e)  
    showToast("Something went wrong")  
}  
}  
}.getOrNull()  
}
```

4. Result Wrapper Pattern

Concept

Instead of returning nullable types or throwing exceptions, wrap results in a container that explicitly represents success or failure.

Basic Result Wrapper (Kotlin)

```
sealed class Result<out T> {  
    data class Success<T>(val data: T) : Result<T>()  
    data class Error(val exception: Throwable) : Result<Nothing>()  
    object Loading : Result<Nothing>()  
}
```

// Helper functions

```
fun <T> Result<T>.getOrNull(): T? = when (this) {  
    is Result.Success -> data  
    is Result.Error -> null  
    is Result.Loading -> null  
}
```

```
fun <T> Result<T>.isSuccess(): Boolean = this is Result.Success  
fun <T> Result<T>.isError(): Boolean = this is Result.Error
```

Usage Example (Kotlin)

```
// Repository  
class UserRepository {  
    suspend fun getUser(id: String): Result<User> {  
        return try {  
            val response = api.getUser(id)  
            if (response.isSuccessful && response.body() != null) {
```

```

Result.Success(response.body()!!)
} else {
Result.Error(HttpException(response))
}
} catch (e: IOException) {
Result.Error(e)
} catch (e: Exception) {
Result.Error(e)
}
}
}
}

```

// ViewModel

```

class UserViewModel(private val repository: UserRepository) :
ViewModel() {

```

```

    private val _userState = MutableStateFlow<Result<User>>(Result.Loading)
    val userState: StateFlow<Result<User>> = _userState.asStateFlow()

```

```

    fun loadUser(id: String) = viewModelScope.launch {
        _userState.value = Result.Loading
        _userState.value = repository.getUser(id)
    }

```

```

}

```

// Activity

```

class UserActivity : AppCompatActivity() {

```

```

    private val viewModel: UserViewModel by viewModels()

```

```

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

```

```

        lifecycleScope.launch {
            viewModel.userState.collect { result ->
                when (result) {
                    is Result.Loading -> {

```

```

        progressBar.visibility = View.VISIBLE
        contentLayout.visibility = View.GONE
        errorLayout.visibility = View.GONE
    }
    is Result.Success -> {
        progressBar.visibility = View.GONE
        contentLayout.visibility = View.VISIBLE
        errorLayout.visibility = View.GONE
        displayUser(result.data)
    }
    is Result.Error -> {
        progressBar.visibility = View.GONE
        contentLayout.visibility = View.GONE
        errorLayout.visibility = View.VISIBLE
        showError(result.exception.message ?: "Unknown error")
    }
}
}
}
}
}

```

```

}

```

Java Result Wrapper

// Result class

```

public abstract class Result<T> {

```

```

    // Success case

```

```

    public static final class Success<T> extends Result<T> {
        private final T data;

```

```

        public Success(T data) {
            this.data = data;
        }

```

```

        public T getData() {
            return data;
        }
    }
}

```

```

    }
}

// Error case
public static final class Error<T> extends Result<T> {
    private final Throwable exception;

    public Error(Throwable exception) {
        this.exception = exception;
    }

    public Throwable getException() {
        return exception;
    }
}

// Loading case
public static final class Loading<T> extends Result<T> {
    public Loading() {}
}

// Private constructor
private Result() {}
}

```

```

// Repository
public class UserRepository {

```

```

    public LiveData<Result<User>> getUser(String id) {
        MutableLiveData<Result<User>> result = new MutableLiveData<>();
        result.setValue(new Result.Loading<>());

        executor.execute() -> {
            try {
                Response<User> response = api.getUser(id).execute();
                if (response.isSuccessful() && response.body() != null) {
                    result.postValue(new Result.Success<>(response.body()));
                }
            } catch (Exception e) {
                result.postValue(new Result.Error<>(e));
            }
        }
    }
}

```

```

        } else {
            result.postValue(new Result.Error<>(
                new HttpException(response)
            ));
        }
    } catch (IOException e) {
        result.postValue(new Result.Error<>(e));
    } catch (Exception e) {
        result.postValue(new Result.Error<>(e));
    }
    });

    return result;
}

```

```

}

```

// Activity

public class UserActivity extends AppCompatActivity {

```

    private UserViewModel viewModel;

```

```

    @Override

```

```

    protected void onCreate(Bundle savedInstanceState) {

```

```

        super.onCreate(savedInstanceState);

```

```

        setContentView(R.layout.activity_user);

```

```

        viewModel = new ViewModelProvider(this).get(UserViewModel.class);

```

```

        viewModel.getUserState().observe(this, result -> {

```

```

            if (result instanceof Result.Loading) {

```

```

                progressBar.setVisibility(View.VISIBLE);

```

```

                contentLayout.setVisibility(View.GONE);

```

```

                errorLayout.setVisibility(View.GONE);

```

```

            } else if (result instanceof Result.Success) {

```

```

                progressBar.setVisibility(View.GONE);

```

```

                contentLayout.setVisibility(View.VISIBLE);

```

```

                errorLayout.setVisibility(View.GONE);
            }
        }
    }
}

```

```

        User user = ((Result.Success<User>) result).getData();
        displayUser(user);
    } else if (result instanceof Result.Error) {
        progressBar.setVisibility(View.GONE);
        contentLayout.setVisibility(View.GONE);
        errorLayout.setVisibility(View.VISIBLE);
        Throwable error = ((Result.Error<User>) result).getException();
        showError(error.getMessage());
    }
});
}

}

```

5. Sealed Class Approach

Resource Sealed Class (Extended Pattern)

Kotlin:

```

sealed class Resource<out T> {
    data class Success<T>(val data: T) : Resource<T>()
    data class Error(
        val message: String,
        val exception: Throwable? = null
    ) : Resource<Nothing>()
    object Loading : Resource<Nothing>()
}

```

```

// Helper methods
fun isSuccess(): Boolean = this is Success
fun isError(): Boolean = this is Error
fun isLoading(): Boolean = this is Loading

fun getOrNull(): T? = when (this) {
    is Success -> data
    else -> null
}

fun getOrThrow(): T = when (this) {

```

```

    is Success -> data
    is Error -> throw exception ?: Exception(message)
    is Loading -> throw IllegalStateException("Still loading")
}

fun <R> map(transform: (T) -> R): Resource<R> = when (this) {
    is Success -> Success(transform(data))
    is Error -> Error(message, exception)
    is Loading -> Loading
}

```

```

}

```

Java:

```

public abstract class Resource<T> {

```

```

    public static final class Success<T> extends Resource<T> {
        private final T data;

        public Success(T data) {
            this.data = data;
        }

        public T getData() {
            return data;
        }
    }
}

```

```

    public static final class Error<T> extends Resource<T> {
        private final String message;
        private final Throwable exception;

        public Error(String message, Throwable exception) {
            this.message = message;
            this.exception = exception;
        }

        public String getMessage() {

```



```

        return message;
    }

    public Throwable getException() {
        return exception;
    }
}

public static final class Loading<T> extends Resource<T> {
    public Loading() {}

    // Helper methods
    public boolean isSuccess() {
        return this instanceof Success;
    }

    public boolean isError() {
        return this instanceof Error;
    }

    public boolean isLoading() {
        return this instanceof Loading;
    }

    public T getOrNull() {
        if (this instanceof Success) {
            return ((Success<T>) this).getData();
        }
        return null;
    }
}

```

Complete Example: Weather App

Kotlin:

```
// ===== DATA LAYER =====
```

```
data class Weather(  
    val city: String,  
    val temperature: Double,  
    val condition: String  
)
```

```
// Repository
```

```
class WeatherRepository(private val api: WeatherApi) {
```

```
    suspend fun getWeather(city: String): Resource<Weather> {  
        return try {  
            // Emit loading can be done in ViewModel  
            val response = api.getWeather(city)  
  
            if (response.isSuccessful && response.body() != null) {  
                Resource.Success(response.body()!!)  
            } else {  
                Resource.Error(  
                    message = "Failed to fetch weather: ${response.code()}",  
                    exception = HttpException(response)  
                )  
            }  
        } catch (e: IOException) {  
            Resource.Error(  
                message = "No internet connection",  
                exception = e  
            )  
        } catch (e: HttpException) {  
            Resource.Error(  
                message = "Server error: ${e.code()}",  
                exception = e  
            )  
        } catch (e: Exception) {  
            Resource.Error(  
                message = "Unknown error: ${e.message}",  
                exception = e  
            )  
        }  
    }
```

```

        message = "Unknown error: ${e.message}",
        exception = e
    )
}
}

// With caching
fun getWeatherWithCache(city: String): Flow<Resource<Weather>> = flow {
    emit(Resource.Loading)

    // Emit cached data first
    val cachedWeather = weatherDao.getWeather(city)
    if (cachedWeather != null) {
        emit(Resource.Success(cachedWeather))
    }

    // Fetch fresh data
    when (val result = getWeather(city)) {
        is Resource.Success -> {
            weatherDao.insert(result.data)
            emit(result)
        }
        is Resource.Error -> {
            if (cachedWeather == null) {
                emit(result)
            }
        }
        is Resource.Loading -> {} // Already emitted
    }
}.flowOn(Dispatchers.IO)

```

```

}

```

```

// ===== PRESENTATION LAYER =====

```

```

@HiltViewModel

```

```

class WeatherViewModel @Inject constructor(

```

```
private val repository: WeatherRepository
): ViewModel() {
```

```
    private val _weatherState = MutableStateFlow<Resource<Weather>>(Resource.Loading)
    val weatherState: StateFlow<Resource<Weather>> = _weatherState.asStateFlow()
```

```
    fun loadWeather(city: String) {
        viewModelScope.launch {
            _weatherState.value = Resource.Loading
            _weatherState.value = repository.getWeather(city)
        }
    }
}
```

```
// With caching
fun loadWeatherWithCache(city: String) {
    viewModelScope.launch {
        repository.getWeatherWithCache(city).collect { resource ->
            _weatherState.value = resource
        }
    }
}
```

```
}
```

```
// ===== UI LAYER (Compose) =====
```

```
@Composable
```

```
fun WeatherScreen(viewModel: WeatherViewModel =
hiltViewModel()) {
    val weatherState by viewModel.weatherState.collectAsState()
```

```
    var cityInput by remember { mutableStateOf("") }

    Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {
        OutlinedTextField(
            value = cityInput,
            onChange = { cityInput = it },
            label = { Text("Enter city") },
```

```

        modifier = Modifier.fillMaxWidth()
    )

    Button(
        onClick = { viewModel.loadWeather(cityInput) },
        modifier = Modifier.fillMaxWidth().padding(top = 8.dp)
    ) {
        Text("Search")
    }

    Spacer(modifier = Modifier.height(16.dp))

    when (val state = weatherState) {
        is Resource.Loading -> {
            CircularProgressIndicator(
                modifier = Modifier.align(Alignment.CenterHorizontally)
            )
        }

        is Resource.Success -> {
            WeatherCard(weather = state.data)
        }

        is Resource.Error -> {
            ErrorCard(
                message = state.message,
                onRetry = { viewModel.loadWeather(cityInput) }
            )
        }
    }
}

}

@Composable
fun WeatherCard(weather: Weather) {
    Card(
        modifier = Modifier.fillMaxWidth(),

```

```

elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
) {
Column(
modifier = Modifier.padding(16.dp),
horizontalAlignment = Alignment.CenterHorizontally
) {
Text(
text = weather.city,
style = MaterialTheme.typography.headlineMedium
)
Text(
text = "${weather.temperature}°C",
style = MaterialTheme.typography.displayLarge
)
Text(
text = weather.condition,
style = MaterialTheme.typography.bodyLarge
)
}
}
}

```

```

@Composable
fun ErrorCard(message: String, onRetry: () -> Unit) {
Card(
modifier = Modifier.fillMaxWidth(),
colors = CardDefaults.cardColors(
containerColor = MaterialTheme.colorScheme.errorContainer
)
) {
Column(
modifier = Modifier.padding(16.dp),
horizontalAlignment = Alignment.CenterHorizontally
) {
Icon(
imageVector = Icons.Default.Error,
contentDescription = null,
tint = MaterialTheme.colorScheme.error,
modifier = Modifier.size(48.dp)
)

```

```

        Spacer(modifier = Modifier.height(8.dp))

        Text(
            text = message,
            style = MaterialTheme.typography.bodyLarge,
            textAlign = TextAlign.Center
        )

        Spacer(modifier = Modifier.height(8.dp))

        Button(onClick = onRetry) {
            Text("Retry")
        }
    }
}

```

```

}

```

Java:

```

// ===== DATA LAYER =====

```

```

public class Weather {
    private String city;
    private double temperature;
    private String condition;

```

```

    // Constructors, getters, setters

```

```

}

```

```

// Repository

```

```

public class WeatherRepository {

```

```

    private final WeatherApi api;
    private final ExecutorService executor;

```

```

    public WeatherRepository(WeatherApi api) {

```

```

        this.api = api;
        this.executor = Executors.newSingleThreadExecutor();
    }

    public LiveData<Resource<Weather>> getWeather(String city) {
        MutableLiveData<Resource<Weather>> result = new MutableLiveData<>();
        result.setValue(new Resource.Loading<>());

        executor.execute() -> {
            try {
                Response<Weather> response = api.getWeather(city).execute();

                if (response.isSuccessful() && response.body() != null) {
                    result.postValue(new Resource.Success<>(response.body()));
                } else {
                    result.postValue(new Resource.Error<>(
                        "Failed to fetch weather: " + response.code(),
                        new HttpException(response)
                    ));
                }
            } catch (IOException e) {
                result.postValue(new Resource.Error<>(
                    "No internet connection",
                    e
                ));
            } catch (Exception e) {
                result.postValue(new Resource.Error<>(
                    "Unknown error: " + e.getMessage(),
                    e
                ));
            }
        });

        return result;
    }
}

```



```
// ===== PRESENTATION LAYER =====
```

```
public class WeatherViewModel extends ViewModel {
```

```
    private final WeatherRepository repository;
    private final MutableLiveData<Resource<Weather>> weatherState;

    public WeatherViewModel(WeatherRepository repository) {
        this.repository = repository;
        this.weatherState = new MutableLiveData<>(new Resource.Loading<>());
    }

    public LiveData<Resource<Weather>> getWeatherState() {
        return weatherState;
    }

    public void loadWeather(String city) {
        weatherState.setValue(new Resource.Loading<>());

        repository.getWeather(city).observeForever(resource -> {
            weatherState.setValue(resource);
        });
    }
}
```

```
}
```

```
// ===== UI LAYER (Activity) =====
```

```
public class WeatherActivity extends AppCompatActivity {
```

```
    private WeatherViewModel viewModel;
    private EditText cityInput;
    private Button searchButton;
    private ProgressBar progressBar;
    private CardView weatherCard;
    private CardView errorCard;
    private TextView cityText;
    private TextView temperatureText;
```

```

private TextView conditionText;
private TextView errorMessage;
private Button retryButton;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_weather);

    initViews();
    initViewModel();
    setupListeners();
}

private void initViews() {
    cityInput = findViewById(R.id.cityInput);
    searchButton = findViewById(R.id.searchButton);
    progressBar = findViewById(R.id.progressBar);
    weatherCard = findViewById(R.id.weatherCard);
    errorCard = findViewById(R.id.errorCard);
    cityText = findViewById(R.id.cityText);
    temperatureText = findViewById(R.id.temperatureText);
    conditionText = findViewById(R.id.conditionText);
    errorMessage = findViewById(R.id.errorMessage);
    retryButton = findViewById(R.id.retryButton);
}

private void initViewModel() {
    viewModel = new ViewModelProvider(this).get(WeatherViewModel.class);

    viewModel.getWeatherState().observe(this, resource -> {
        if (resource instanceof Resource.Loading) {
            showLoading();
        } else if (resource instanceof Resource.Success) {
            Weather weather = ((Resource.Success<Weather>) resource).getData();
            showWeather(weather);
        } else if (resource instanceof Resource.Error) {
            Resource.Error<Weather> error = (Resource.Error<Weather>) resource

```

```

        showError(error.getMessage());
    }
});
}

private void setupListeners() {
    searchButton.setOnClickListener(v -> {
        String city = cityInput.getText().toString().trim();
        if (!city.isEmpty()) {
            viewModel.loadWeather(city);
        }
    });

    retryButton.setOnClickListener(v -> {
        String city = cityInput.getText().toString().trim();
        viewModel.loadWeather(city);
    });
}

private void showLoading() {
    progressBar.setVisibility(View.VISIBLE);
    weatherCard.setVisibility(View.GONE);
    errorCard.setVisibility(View.GONE);
}

private void showWeather(Weather weather) {
    progressBar.setVisibility(View.GONE);
    weatherCard.setVisibility(View.VISIBLE);
    errorCard.setVisibility(View.GONE);

    cityText.setText(weather.getCity());
    temperatureText.setText(weather.getTemperature() + "°C");
    conditionText.setText(weather.getCondition());
}

private void showError(String message) {
    progressBar.setVisibility(View.GONE);
    weatherCard.setVisibility(View.GONE);
}

```

```
        errorCard.setVisibility(View.VISIBLE);

        errorMessage.setText(message);
    }
}
```

6. Flow Error Handling

Flow Operators for Error Handling

Kotlin Flow provides several operators:

- **catch** - Catches upstream exceptions
- **onEach** - Performs actions on each emission
- **onCompletion** - Executes when flow completes
- **retry** - Retries on failure
- **retryWhen** - Conditional retry

Basic Flow Error Handling

Kotlin:

```
fun fetchUsers(): Flow<List<User>> = flow {
    val users = api.getUsers()
    emit(users)
}.catch { e ->
    Log.e("Flow", "Error fetching users", e)
    emit(emptyList()) // Emit default value
}.flowOn(Dispatchers.IO)

// Usage
lifecycleScope.launch {
    fetchUsers().collect { users ->
        adapter.submitList(users)
    }
}
```

Flow with Resource Wrapper

Kotlin:

```
fun fetchUsersWithState(): Flow<Resource<List<User>>> = flow {  
    emit(Resource.Loading)
```

```
    try {  
        val users = api.getUsers()  
        emit(Resource.Success(users))  
    } catch (e: IOException) {  
        emit(Resource.Error("No internet connection", e))  
    } catch (e: Exception) {  
        emit(Resource.Error("Unknown error", e))  
    }  
}
```

```
}.flowOn(Dispatchers.IO)
```

Retry Strategies

Simple retry:

```
fun fetchData(): Flow<Data> = flow {  
    val data = api.getData()  
    emit(data)  
}.retry(3) { cause ->  
    // Retry on network errors only  
    cause is IOException  
}.catch { e ->  
    Log.e("Flow", "Failed after retries", e)  
    emit(Data.empty())  
}.flowOn(Dispatchers.IO)
```

Retry with delay:

```
fun fetchDataWithRetry(): Flow<Data> = flow {  
    val data = api.getData()  
    emit(data)  
}.retryWhen { cause, attempt ->  
    if (cause is IOException && attempt < 3) {  
        delay(2000 * (attempt + 1)) // Exponential backoff  
        true // Retry
```

```

} else {
false // Don't retry
}
}.catch { e ->
Log.e("Flow", "Failed after retries", e)
emit(Data.empty())
}.flowOn(Dispatchers.IO)

```

Complex Flow Error Handling

Kotlin:

```

class UserRepository(
private val api: UserApi,
private val dao: UserDao
){

```

```

fun getUsers(): Flow<Resource<List<User>>> = flow {
    emit(Resource.Loading)

    // Try to emit cached data first
    val cachedUsers = dao.getAllUsers().first()
    if (cachedUsers.isNotEmpty()) {
        emit(Resource.Success(cachedUsers))
    }

    // Fetch fresh data
    val users = api.getUsers()
    dao.insertAll(users)
    emit(Resource.Success(users))

}.catch { e ->
    when (e) {
        is IOException -> {
            Log.e("Repository", "Network error", e)
            // If we have cache, don't emit error
            val cachedUsers = dao.getAllUsers().first()
            if (cachedUsers.isEmpty()) {
                emit(Resource.Error("No internet connection", e))
            }
        }
    }
}

```

```

    }
  }
  is HttpException -> {
    Log.e("Repository", "Server error: ${e.code()}", e)
    emit(Resource.Error("Server error", e))
  }
  else -> {
    Log.e("Repository", "Unknown error", e)
    emit(Resource.Error("Something went wrong", e))
  }
}
}.flowOn(Dispatchers.IO)

```

```

}

```

Flow Error Handling Patterns

Pattern 1: Catch and emit default:

```

flow {
  emit(api.getData())
}.catch { e ->
  Log.e("TAG", "Error", e)
  emit(Data.empty())
}

```

Pattern 2: Catch and rethrow:

```

flow {
  emit(api.getData())
}.catch { e ->
  Log.e("TAG", "Error", e)
  throw e // Propagate error
}

```

Pattern 3: Catch and emit error state:

```

flow {
  emit(Resource.Loading)
  emit(Resource.Success(api.getData()))
}.catch { e ->
  emit(Resource.Error(e.message, e))
}

```

Pattern 4: Multiple operators:

```
api.getUsersFlow()
    .onEach { users ->
        Log.d("TAG", "Received ${users.size} users")
    }
    .catch { e ->
        Log.e("TAG", "Error", e)
        emit(emptyList())
    }
    .onCompletion { cause ->
        if (cause == null) {
            Log.d("TAG", "Flow completed successfully")
        } else {
            Log.e("TAG", "Flow completed with error", cause)
        }
    }
    .collect { users ->
        updateUI(users)
    }
```

7. Global Exception Handler

Concept

A global exception handler catches ALL uncaught exceptions in your app, preventing crashes and allowing custom crash reporting.

Basic Implementation (Kotlin)

```
class GlobalExceptionHandler(
    private val context: Context,
    private val defaultHandler: Thread.UncaughtExceptionHandler?
) : Thread.UncaughtExceptionHandler {
```

```
    override fun uncaughtException(thread: Thread, exception: Throwable) {
        try {
            // Log the exception
            Log.e("GlobalHandler", "Uncaught exception", exception)
        } catch (e: Exception) {
            // Log the exception
            Log.e("GlobalHandler", "Uncaught exception", e)
        }
    }
}
```



```

        // Save crash report to file
        saveCrashReport(exception)

        // Send to crash reporting service (Firebase Crashlytics, Sentry, etc.)
        sendToCrashReporting(exception)

        // Show user-friendly message (optional)
        showCrashDialog(exception)

    } catch (e: Exception) {
        Log.e("GlobalHandler", "Error in exception handler", e)
    } finally {
        // Call default handler to let system handle the crash
        defaultHandler?.uncaughtException(thread, exception)
    }
}

private fun saveCrashReport(exception: Throwable) {
    try {
        val timestamp = SimpleDateFormat("yyyy-MM-dd_HH-mm-ss", Locale.US)
            .format(Date())
        val filename = "crash_$timestamp.txt"

        val crashReport = buildString {
            append("Timestamp: ${Date()}\n")
            append("Thread: ${Thread.currentThread().name}\n")
            append("Exception: ${exception.javaClass.simpleName}\n")
            append("Message: ${exception.message}\n\n")
            append("Stack Trace:\n")
            append(exception.stackTraceToString())
        }

        val file = File(context.filesDir, filename)
        file.writeText(crashReport)

        Log.d("GlobalHandler", "Crash report saved: $filename")
    } catch (e: Exception) {

```

```

        Log.e("GlobalHandler", "Failed to save crash report", e)
    }
}

private fun sendToCrashReporting(exception: Throwable) {
    // Example: Firebase Crashlytics
    // FirebaseCrashlytics.getInstance().recordException(exception)

    // Example: Sentry
    // Sentry.captureException(exception)
}

private fun showCrashDialog(exception: Throwable) {
    try {
        val intent = Intent(context, CrashActivity::class.java).apply {
            putExtra("error_message", exception.message)
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP)
        }
        context.startActivity(intent)
    } catch (e: Exception) {
        Log.e("GlobalHandler", "Failed to show crash dialog", e)
    }
}

}

```

```

// Application class
class MyApplication : Application() {

```

```

    override fun onCreate() {
        super.onCreate()

        // Install global exception handler
        val defaultHandler = Thread.getDefaultUncaughtExceptionHandler()
        val globalHandler = GlobalExceptionHandler(this, defaultHandler)
    }
}

```

```
Thread.setDefaultUncaughtExceptionHandler(globalHandler)
}
```

```
}
```

Java Implementation

public class GlobalExceptionHandler implements
Thread.UncaughtExceptionHandler {

```
private final Context context;
private final Thread.UncaughtExceptionHandler defaultHandler;

public GlobalExceptionHandler(Context context) {
    this.context = context;
    this.defaultHandler = Thread.getDefaultUncaughtExceptionHandler();
}

@Override
public void uncaughtException(Thread thread, Throwable exception) {
    try {
        // Log the exception
        Log.e("GlobalHandler", "Uncaught exception", exception);

        // Save crash report
        saveCrashReport(exception);

        // Send to crash reporting
        sendToCrashReporting(exception);

        // Show crash dialog
        showCrashDialog(exception);

    } catch (Exception e) {
        Log.e("GlobalHandler", "Error in exception handler", e);
    } finally {
        // Call default handler
        if (defaultHandler != null) {
```

```

        defaultHandler.uncaughtException(thread, exception);
    }
}

private void saveCrashReport(Throwable exception) {
    try {
        SimpleDateFormat sdf = new SimpleDateFormat(
            "yyyy-MM-dd_HH-mm-ss",
            Locale.US
        );
        String timestamp = sdf.format(new Date());
        String filename = "crash_" + timestamp + ".txt";

        StringBuilder crashReport = new StringBuilder();
        crashReport.append("Timestamp: ").append(new Date()).append("\n");
        crashReport.append("Thread: ").append(Thread.currentThread().getNam
        crashReport.append("Exception: ").append(exception.getClass().getSimpl
        crashReport.append("Message: ").append(exception.getMessage()).appen
        crashReport.append("Stack Trace:\n");

        StringWriter sw = new StringWriter();
        PrintWriter pw = new PrintWriter(sw);
        exception.printStackTrace(pw);
        crashReport.append(sw.toString());

        File file = new File(context.getFilesDir(), filename);
        FileWriter writer = new FileWriter(file);
        writer.write(crashReport.toString());
        writer.close();

        Log.d("GlobalHandler", "Crash report saved: " + filename);
    } catch (Exception e) {
        Log.e("GlobalHandler", "Failed to save crash report", e);
    }
}

private void sendToCrashReporting(Throwable exception) {

```

```

        // Firebase Crashlytics
        // FirebaseCrashlytics.getInstance().recordException(exception);
    }

    private void showCrashDialog(Throwable exception) {
        try {
            Intent intent = new Intent(context, CrashActivity.class);
            intent.putExtra("error_message", exception.getMessage());
            intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
            intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
            context.startActivity(intent);
        } catch (Exception e) {
            Log.e("GlobalHandler", "Failed to show crash dialog", e);
        }
    }
}

// Application class
public class MyApplication extends Application {

    @Override
    public void onCreate() {
        super.onCreate();

        // Install global exception handler
        GlobalExceptionHandler handler = new GlobalExceptionHandler(this);
        Thread.setDefaultUncaughtExceptionHandler(handler);
    }
}

```

Crash Activity (User-Friendly Error Screen)

Kotlin:

```
class CrashActivity : AppCompatActivity() {
```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_crash)

    val errorMessage = intent.getStringExtra("error_message") ?: "Unknown error"

    errorTextView.text = "We're sorry, the app has encountered an error:\n\n$errorMessage"

    restartButton.setOnClickListener {
        val intent = Intent(this, MainActivity::class.java)
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP)
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        startActivity(intent)
        finish()
    }

    closeButton.setOnClickListener {
        finishAffinity() // Close all activities
    }
}
}

```

8. Best Practices

1. Catch Specific Exceptions

✗ BAD:

```

try {
    riskyOperation()
} catch (e: Exception) {
    // Too generic!
    Log.e("TAG", "Error", e)
}

```

✓ GOOD:

```

try {
    riskyOperation()
} catch (e: IOException) {
    // Handle specific exception
}

```

```

} catch (e: IOException) {
    Log.e("TAG", "Network error", e)
    showToast("Check your internet connection")
} catch (e: NumberFormatException) {
    Log.e("TAG", "Invalid number", e)
    showToast("Please enter a valid number")
} catch (e: Exception) {
    Log.e("TAG", "Unexpected error", e)
    showToast("Something went wrong")
}

```

2. Don't Swallow Exceptions

✗ BAD:

```

try {
    importantOperation()
} catch (e: Exception) {
    // Silent failure - BAD!
}

```

✓ GOOD:

```

try {
    importantOperation()
} catch (e: Exception) {
    Log.e("TAG", "Important operation failed", e)
    // Notify user
    showError("Operation failed: ${e.message}")
    // Report to crash analytics
    reportError(e)
}

```

3. Provide User-Friendly Messages

✗ BAD:

```

catch (e: Exception) {
    Toast.makeText(context, e.message, LENGTH_LONG).show()
    // Shows: "java.net.SocketTimeoutException: timeout"
}

```

✓ GOOD:

```

catch (e: IOException) {

```

```
Toast.makeText(  
context,  
"Cannot connect to server. Please check your internet.",  
LENGTH_LONG  
)  
.show()  
}
```

4. Use Finally for Cleanup

✓ **GOOD:**

```
var reader: BufferedReader? = null  
try {  
    reader = File("data.txt").bufferedReader()  
    val content = reader.readText()  
    processContent(content)  
} catch (e: IOException) {  
    Log.e("TAG", "File error", e)  
} finally {  
    reader?.close() // Always close resources  
}
```

```
// Or better: use 'use' function (Kotlin)  
File("data.txt").bufferedReader().use { reader ->  
    val content = reader.readText()  
    processContent(content)  
}
```

5. Log with Context

✗ **BAD:**

```
catch (e: Exception) {  
    Log.e("TAG", "Error") // No details!  
}
```

✓ **GOOD:**

```
catch (e: Exception) {  
    Log.e("UserRepository", "Failed to load user $userId", e)  
    // Includes context, exception details, and stack trace  
}
```


6. Don't Use Try-Catch for Flow Control

✗ BAD:

```
fun isValidNumber(text: String): Boolean {  
    return try {  
        text.toInt()  
        true  
    } catch (e: NumberFormatException) {  
        false  
    }  
}
```

✓ GOOD:

```
fun isValidNumber(text: String): Boolean {  
    return text.toIntOrNull() != null  
}
```

7. Handle Errors at Appropriate Level

✗ BAD:

```
// Low-level function showing UI  
fun loadFromDatabase(): User? {  
    return try {  
        database.getUser()  
    } catch (e: SQLException) {  
        Toast.makeText(context, "Database error", LENGTH_SHORT).show()  
        null  
    }  
}
```

✓ GOOD:

```
// Repository: Propagate error  
suspend fun loadUser(): Result<User> {  
    return try {  
        val user = database.getUser()  
        Result.Success(user)  
    } catch (e: SQLException) {  
        Result.Error(e)  
    }  
}
```

```
// ViewModel: Transform error
fun loadUser() {
    viewModelScope.launch {
        when (val result = repository.loadUser()) {
            is Result.Success -> _user.value = result.data
            is Result.Error -> _error.value = "Failed to load user"
        }
    }
}
```

```
// Activity: Show UI
lifecycleScope.launch {
    viewModel.error.collect { message ->
        Toast.makeText(this@MainActivity, message,
            LENGTH_SHORT).show()
    }
}
```

8. Test Error Paths

✓ **GOOD:**

```
@Test
fun loadUser should return error on network failure() = runTest {
    // Mock repository to throw exception
    whenever(repository.getUser(any())).thenThrow(IOException())
```

```
viewModel.loadUser("123")
```

```
val state = viewModel.userState.value
assertTrue(state is Resource.Error)
assertEquals("No internet connection", (state as Resource.Error).message)
```

```
}
```

9. Real-World Examples

Example 1: Login Flow with Error Handling

Kotlin:

```
// ===== DATA LAYER =====

sealed class LoginError {
    object InvalidCredentials : LoginError()
    object NetworkError : LoginError()
    object ServerError : LoginError()
    data class UnknownError(val message: String) : LoginError()
}

sealed class LoginResult {
    data class Success(val userId: String, val token: String) : LoginResult()
    data class Failure(val error: LoginError) : LoginResult()
}

class LoginRepository(private val api: AuthApi) {

    suspend fun login(username: String, password: String): LoginResult {
        return try {
            val response = api.login(LoginRequest(username, password))

            when (response.code()) {
                200 -> {
                    val body = response.body()!!
                    LoginResult.Success(body.userId, body.token)
                }
                401 -> LoginResult.Failure(LoginError.InvalidCredentials)
                500 -> LoginResult.Failure(LoginError.ServerError)
                else -> LoginResult.Failure(
                    LoginError.UnknownError("HTTP ${response.code()}")
                )
            }
        } catch (e: IOException) {
            LoginResult.Failure(LoginError.NetworkError)
        } catch (e: Exception) {
            LoginResult.Failure(LoginError.UnknownError(e.message))
        }
    }
}
```

```
        LoginResult.Failure(LoginError.UnknownError(e.message ?: "Unknown")
    }
}
```

```
}
```

```
// ===== PRESENTATION LAYER =====
```

```
sealed class LoginUiState {
    object Idle : LoginUiState()
    object Loading : LoginUiState()
    data class Success(val userId: String) : LoginUiState()
    data class Error(val message: String) : LoginUiState()
}
```

```
@HiltViewModel
class LoginViewModel @Inject constructor(
    private val repository: LoginRepository
) : ViewModel() {
```

```
    private val _uiState = MutableStateFlow<LoginUiState>(LoginUiState.Idle)
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()
```

```
    fun login(username: String, password: String) {
        // Validation
        if (username.isEmpty()) {
            _uiState.value = LoginUiState.Error("Username cannot be empty")
            return
        }
    }
```

```
    if (password.isEmpty()) {
        _uiState.value = LoginUiState.Error("Password cannot be empty")
        return
    }
```

```
    // Perform login
    viewModelScope.launch {
        _uiState.value = LoginUiState.Loading
    }
```

```

        when (val result = repository.login(username, password)) {
            is LoginResult.Success -> {
                _uiState.value = LoginUiState.Success(result.userId)
            }
            is LoginResult.Failure -> {
                val message = when (result.error) {
                    is LoginError.InvalidCredentials ->
                        "Invalid username or password"
                    is LoginError.NetworkError ->
                        "No internet connection. Please try again."
                    is LoginError.ServerError ->
                        "Server error. Please try again later."
                    is LoginError.UnknownError ->
                        "Login failed: ${result.error.message}"
                }
                _uiState.value = LoginUiState.Error(message)
            }
        }
    }
}

```

```

}

```

```

// ===== UI LAYER =====

```

@Composable

```

fun LoginScreen(viewModel: LoginViewModel = hiltViewModel()) {
    val uiState by viewModel.uiState.collectAsState()

```

```

        var username by remember { mutableStateOf("") }
        var password by remember { mutableStateOf("") }

```

```

        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(16.dp),
            verticalArrangement = Arrangement.Center

```

```

) {
    OutlinedTextField(
        value = username,
        onChange = { username = it },
        label = { Text("Username") },
        modifier = Modifier.fillMaxWidth()
    )

    Spacer(modifier = Modifier.height(8.dp))

    OutlinedTextField(
        value = password,
        onChange = { password = it },
        label = { Text("Password") },
        visualTransformation = PasswordVisualTransformation(),
        modifier = Modifier.fillMaxWidth()
    )

    Spacer(modifier = Modifier.height(16.dp))

    when (val state = uiState) {
        is LoginUiState.Loading -> {
            CircularProgressIndicator(
                modifier = Modifier.align(Alignment.CenterHorizontally)
            )
        }

        is LoginUiState.Error -> {
            Text(
                text = state.message,
                color = MaterialTheme.colorScheme.error,
                modifier = Modifier.padding(bottom = 8.dp)
            )
        }

        is LoginUiState.Success -> {
            Text(
                text = "Login successful!",

```

```

        color = MaterialTheme.colorScheme.primary
    )
}

else -> {}
}

Button(
    onClick = { viewModel.login(username, password) },
    modifier = Modifier.fillMaxWidth(),
    enabled = uiState !is LoginUiState.Loading
) {
    Text("Login")
}
}
}
}

```

Example 2: File Upload with Error Recovery

Kotlin:

```

sealed class UploadState {
    object Idle : UploadState()
    data class Uploading(val progress: Int) : UploadState()
    object Success : UploadState()
    data class Error(val message: String, val canRetry: Boolean) :
        UploadState()
}

```

```

class FileUploadRepository(private val api: UploadApi) {

```

```

    suspend fun uploadFile(file: File): Flow<UploadState> = flow {
        emit(UploadState.Uploading(0))

        try {
            val requestBody = file.asRequestBody("image/*".toMediaTypeOrNull())
            val multipartBody = MultipartBody.Part.createFormData(
                "file",

```

```

        file.name,
        requestBody
    )

    api.uploadFile(multipartBody).let { response ->
        if (response.isSuccessful) {
            emit(UploadState.Success)
        } else {
            emit(UploadState.Error(
                message = "Upload failed: ${response.code()}",
                canRetry = response.code() in 500..599
            ))
        }
    }
} catch (e: IOException) {
    emit(UploadState.Error(
        message = "Network error. Check your connection.",
        canRetry = true
    ))
} catch (e: Exception) {
    emit(UploadState.Error(
        message = "Upload failed: ${e.message}",
        canRetry = false
    ))
}
}.retry(3) { cause ->
    cause is IOException
}.flowOn(Dispatchers.IO)
}

```

Example 3: Database Operations with Transactions

Kotlin:

```
class TransactionRepository(private val database: AppDatabase) {
```

```

    suspend fun transferFunds(
        fromAccount: String,

```



```

        toAccount: String,
        amount: Double
    ): Result<Unit> = withContext(Dispatchers.IO) {
        try {
            database.withTransaction {
                // Debit from account
                val fromBalance = database.accountDao().getBalance(fromAccount)
                if (fromBalance < amount) {
                    throw InsufficientFundsException("Insufficient balance")
                }
                database.accountDao().updateBalance(fromAccount, fromBalance - amount)

                // Credit to account
                val toBalance = database.accountDao().getBalance(toAccount)
                database.accountDao().updateBalance(toAccount, toBalance + amount)

                // Log transaction
                database.transactionDao().insert(
                    Transaction(from = fromAccount, to = toAccount, amount = amount)
                )
            }

            Result.success(Unit)
        } catch (e: InsufficientFundsException) {
            Result.failure(e)
        } catch (e: SQLException) {
            Log.e("Repository", "Database transaction failed", e)
            Result.failure(Exception("Transaction failed. Please try again."))
        }
    }
}

class InsufficientFundsException(message: String) :
    Exception(message)

```

10. Testing Error Handling

Unit Testing Error Cases

Kotlin:

```
class UserRepositoryTest {
```

```
    @Mock
    lateinit var api: UserApi

    private lateinit var repository: UserRepository

    @Before
    fun setup() {
        MockitoAnnotations.openMocks(this)
        repository = UserRepository(api)
    }

    @Test
    fun `getUser returns error on network failure`() = runTest {
        // Given
        whenever(api.getUser(any())).thenThrow(IOException("Network error"))

        // When
        val result = repository.getUser("123")

        // Then
        assertTrue(result is Resource.Error)
        assertEquals("No internet connection", (result as Resource.Error).message)
    }

    @Test
    fun `getUser returns error on 404`() = runTest {
        // Given
        val response = Response.error<User>(
            404,
            ResponseBody.create(null, "")
        )
```

```

        whenever(api.getUser(any())).thenReturn(response)

        // When
        val result = repository.getUser("123")

        // Then
        assertTrue(result is Resource.Error)
        assertTrue((result as Resource.Error).message.contains("404"))
    }

    @Test
    fun `getUser returns success on valid response`() = runTest {
        // Given
        val user = User("123", "John", "john@test.com")
        val response = Response.success(user)
        whenever(api.getUser(any())).thenReturn(response)

        // When
        val result = repository.getUser("123")

        // Then
        assertTrue(result is Resource.Success)
        assertEquals(user, (result as Resource.Success).data)
    }
}

```

Testing Flow Error Handling

Kotlin:

```
class WeatherViewModelTest {
```

```

    @Mock
    lateinit var repository: WeatherRepository

    private lateinit var viewModel: WeatherViewModel

    @Before

```

```

fun setup() {
    Dispatchers.setMain(StandardTestDispatcher())
    MockitoAnnotations.openMocks(this)
    viewModel = WeatherViewModel(repository)
}

@After
fun tearDown() {
    Dispatchers.resetMain()
}

@Test
fun `loadWeather emits error on failure`() = runTest {
    // Given
    whenever(repository.getWeather(any()))
        .thenReturn(Resource.Error("Network error"))

    // When
    viewModel.loadWeather("Delhi")
    advanceUntilIdle()

    // Then
    val state = viewModel.weatherState.value
    assertTrue(state is Resource.Error)
    assertEquals("Network error", (state as Resource.Error).message)
}

@Test
fun `loadWeather emits loading then success`() = runTest {
    // Given
    val weather = Weather("Delhi", 25.0, "Sunny")
    whenever(repository.getWeather(any()))
        .thenReturn(Resource.Success(weather))

    val states = mutableListOf<Resource<Weather>>()
    val job = launch {
        viewModel.weatherState.collect { states.add(it) }
    }
}

```

```
// When
viewModel.loadWeather("Delhi")
advanceUntilIdle()

// Then
assertTrue(states[0] is Resource.Loading)
assertTrue(states[1] is Resource.Success)
assertEquals(weather, (states[1] as Resource.Success).data)

job.cancel()
}
```

```
}
```

11. Comparison Table

Approach	Pros	Cons	Use Case
Try-Catch	Simple, direct	Verbose, can't distinguish states	Simple operations
Nullable Return	Easy to use	Loss of error info	Non-critical operations
Result Wrapper	Type-safe, clear states	More boilerplate	Repository layer
Sealed Class	Exhaustive when, custom states	Initial setup	ViewModel/UI state
Flow catch	Reactive, composable	Learning curve	Async streams
Global Handler	Catches all crashes	Can't recover gracefully	Last resort

When to Use Each Approach

Scenario	Recommended Approach
Network API calls	Resource sealed class + Flow
Database operations	Try-catch + Result wrapper
File I/O	Try-catch with specific exceptions
Input validation	Return Boolean or sealed class
Background tasks	WorkManager + error callbacks
Real-time updates	Flow with catch operator
Production crash tracking	Global handler + Crashlytics

12. Anti-Patterns to Avoid

Anti-Pattern 1: Empty Catch Block

✗ DON'T:

```
try {  
    riskyOperation()  
} catch (e: Exception) {  
    // Silent failure  
}
```

✓ DO:

```
try {  
    riskyOperation()  
} catch (e: Exception) {  
    Log.e("TAG", "Operation failed", e)  
    reportError(e)  
}
```

Anti-Pattern 2: Catching Throwable

✗ DON'T:

```
try {  
    operation()  
} catch (t: Throwable) {  
    // Catches OutOfMemoryError, StackOverflowError, etc.  
}
```

✓ **DO:**

```
try {  
    operation()  
} catch (e: Exception) {  
    // Only catches Exceptions, not Errors  
}
```

Anti-Pattern 3: Using Try-Catch for Logic

✗ **DON'T:**

```
fun isNumeric(str: String): Boolean {  
    return try {  
        str.toInt()  
        true  
    } catch (e: NumberFormatException) {  
        false  
    }  
}
```

✓ **DO:**

```
fun isNumeric(str: String): Boolean {  
    return str.toIntOrNull() != null  
}
```

Anti-Pattern 4: Nested Try-Catch

✗ **DON'T:**

```
try {  
    val data = fetchData()  
    try {  
        saveData(data)  
        try {  
            notifyUser()  
        } catch (e: Exception) { }  
    } catch (e: Exception) { }  
} catch (e: Exception) { }
```

✓ **DO:**

```
try {  
    val data = fetchData()  
    saveData(data)
```

```
notifyUser()
} catch (e: NetworkException) {
handleNetworkError(e)
} catch (e: StorageException) {
handleStorageError(e)
} catch (e: Exception) {
handleUnknownError(e)
}
```

Anti-Pattern 5: Generic Error Messages

✗ DON'T:

```
catch (e: Exception) {
Toast.makeText(context, "Error occurred", LENGTH_SHORT).show()
}
```

✓ DO:

```
catch (e: IOException) {
Toast.makeText(
context,
"Cannot connect to server. Check your internet connection.",
LENGTH_LONG
).show()
}
```

Anti-Pattern 6: Ignoring Resource Cleanup

✗ DON'T:

```
fun readFile(): String {
val reader = FileReader("data.txt")
return reader.readText() // File never closed!
}
```

✓ DO:

```
fun readFile(): String {
return File("data.txt").bufferedReader().use { reader ->
reader.readText()
} // Automatically closed
}
```

Summary

Key Takeaways

1. **Always handle errors** - Never ignore exceptions
2. **Be specific** - Catch specific exception types
3. **User-friendly messages** - Translate technical errors
4. **Log everything** - Use proper logging for debugging
5. **Use appropriate patterns** - Result wrappers for APIs, try-catch for I/O
6. **Test error paths** - Write tests for failure scenarios
7. **Fail gracefully** - App should continue working when possible

Modern Android Error Handling Stack (2026)

UI Layer

↓ Displays error states

Compose/Views show user-friendly messages

↓

ViewModel Layer

↓ Transforms errors to UI state

Resource/Result sealed classes

↓

Repository Layer

↓ Wraps API/DB calls

Try-catch + Result wrapper

↓

Data Layer

↓ Throws exceptions

Retrofit, Room, File I/O

↓

Global Handler

Catches uncaught exceptions

Logs to Crashlytics/Sentry

Recommended Pattern for New Projects

Use Resource sealed class with Flow:

```
sealed class Resource<out T> {  
    data class Success<T>(val data: T) : Resource<T>()  
}
```

```

data class Error(val message: String, val exception: Throwable?) :
    Resource<Nothing>()
object Loading : Resource<Nothing>()
}

// Repository
fun getData(): Flow<Resource<Data>> = flow {
    emit(Resource.Loading)
    try {
        val data = api.getData()
        emit(Resource.Success(data))
    } catch (e: Exception) {
        emit(Resource.Error("Error: ${e.message}", e))
    }
}

// ViewModel
val dataState = repository.getData()
.stateIn(viewModelScope, SharingStarted.Lazily, Resource.Loading)

// UI
when (val state = dataState.value) {
    is Resource.Loading -> ShowLoading()
    is Resource.Success -> ShowData(state.data)
    is Resource.Error -> ShowError(state.message)
}

```

Perfect for production Android apps with robust error handling!

□

References

[1] Sentry. (2023). How to handle Android exceptions and avoid application crashes. *Sentry Blog*. <https://blog.sentry.io/how-to-handle-android-exceptions-and-avoid-application-crashes/>

[2] Stackademic. (2024). Error Handling in Android: Tips and Best Practices. *Medium*. <https://blog.stackademic.com/error-handling-in-an-droid-tips-and-best-practices-6b6897cd707e>

[3] DotNetSchool. (2025). Android Error Handling And Logging Complete Guide. <https://www.dotnetschool.com/article/web-development/android-tutorial/android-error-handling-and-logging-complete-guide>

[4] Levindo. (2025). Understanding and Applying Exception Handling in Kotlin. *LinkedIn*. <https://www.linkedin.com/pulse/how-use-try-catch-android-understanding-applying-handling-levindo-46ajf>

[5] GitHub. (2021). Result sealed class for kotlin error handling. *Gist*. <https://gist.github.com/saantiaguilera/50af0314906a41bca238492cb21d03ba>

[6] Kotlin. (2026). Sealed classes and interfaces. *Kotlin Documentation*. <https://kotlinlang.org/docs/sealed-classes.html>

[7] ProAndroidDev. (2022). Modeling Retrofit Responses With Sealed Classes and Coroutines. <https://proandroiddev.com/modeling-retrofit-responses-with-sealed-classes-and-coroutines-9d6302077dfe>

[8] GitHub. (2019). Flow error handling and launchIn. *Kotlin Coroutines Issues*. <https://github.com/Kotlin/kotlinx.coroutines/issues/1263>

[9] Stack Overflow. (2012). Ideal way to set global uncaught exception Handler in Android. <https://stackoverflow.com/questions/2764394/ideal-way-to-set-global-uncaught-exception-handler-in-android>

[10] GeeksforGeeks. (2023). Exceptions in Android with Example. <https://www.geeksforgeeks.org/android/exceptions-in-android-with-example/>